

Introduction to Linked List

Linked List

A Linked List is a linear data structure in which elements (called nodes) are connected using pointers. Each node contains:

- A **value** (the actual data)
- A **reference** (or pointer) to the next node (and optionally to the previous node in doubly linked lists)

Types of Linked Lists:

- **value**: the data part
- **next**: a pointer/reference to the next node
- It moves only in one direction (forward)
- **Structure**: [value | next] -> [value | next] -> [value | null]

- **value**: the data
- **next**: pointer to the next node
- **prev**: pointer to the previous node
- Allows bidirectional traversal
- **Structure**: null <- [prev | value | next] <-> [prev | value | next] -> null

Key Terminologies:

- **Head**: The first node of a linked list. It marks the entry point and points to the next node.
- **Tail**: The last node of a linked list. It points to null because there's no node after it.
- **Linked List Representation**: It is typically represented using its head node.

Array vs Linked List — Comparison Table:

Feature	Array	Linked List
Type	Linear data structure where elements are arranged in a sequence using indexes.	Linear data structure where each element (node) points to the next, forming a chain.
Memory Layout	Elements are stored in a contiguous (adjacent) block of memory.	Each node is stored separately and connected via pointers, allowing scattered memory locations.
Size	Size must be known or defined initially. Resizing is expensive if needed later.	Size is dynamic. Nodes can be added or removed without knowing the size in advance.
Data Stored	Only values are stored (e.g., integers, strings, etc.)	Each node stores a value along with one or more pointers (to next and/or previous nodes).
Access Time (Indexing)	Direct access using index (e.g., <code>arr[3]</code>) is fast with time complexity $O(1)$.	Sequential access only. To get a value, you must start at the head and traverse node by node. Time complexity $O(n)$.
Insertion/Deletion	Inserting or deleting in the middle requires	Insertion and deletion are faster, especially at the

Feature	Array	Linked List
	shifting elements. Can be time-consuming.	beginning or end, as it only involves pointer changes.
Memory Efficiency	Efficient memory usage as no extra space is needed for pointers.	Extra memory is used to store pointers (next/prev), making it less memory efficient.

Use Cases: When to Use What?

Use Case	Prefer
Access elements by index fast	Array – Arrays provide direct access to elements using an index, making it very fast ($O(1)$). Ideal when random access is frequent.
Frequent insertions/deletions at head or tail	Linked List – Inserting or removing nodes at the beginning or end is efficient in linked lists since you just change a few pointers ($O(1)$).
Static-sized data with memory-efficient storage	Array – Arrays are more memory-efficient as they don't use extra space for pointers. Best when the number of elements is fixed or predictable.
Unknown size or avoid resizing overhead	Linked List – Since linked lists grow as needed, they are a better choice when the size of the data is not known upfront or changes frequently.
Do a lot of traversal or manipulation	Linked List – When you need to move around, insert or remove nodes repeatedly,

Use Case	Prefer
	linked lists offer better flexibility for such operations.

Implement a singly linked list in JS, C, C++, Java, Python, C#

Problem Statement:

Design and implement a simple singly linked list using basic building blocks like nodes and references (or pointers). The goal is to:

- **Define a node** with a value and a pointer to the next node.
- **Create individual nodes** and link them together to form a list.
- **Traverse the list** from the head and print each node's value.

This helps you understand how data is stored and accessed sequentially in memory using pointers or references.

Example:

- **Input:** Create 3 nodes with values 10, 20, 30 and link them in order.
- **Output:** Print values in sequence: 10 20 30

Approach:

1. Define a `Node` class or structure with a `val` and `next` pointer.
2. Create individual nodes (e.g., node1, node2, node3).
3. Link the nodes: node1.next → node2, node2.next → node3.
4. Set the `head` of the list to the first node.

- 5. Traverse the list using a loop until the current node becomes `null` (or `None`).
- 6. Print the value at each node.

Explanation:

- Each node stores data and a reference to the next node.
- The list is connected by linking nodes via the `next` field.
- The traversal starts from the head and moves node by node until the end.
- This basic structure forms the foundation for many advanced data structures like stacks, queues, and graph adjacency lists.

```
function Node(val) {
  this.val = val;
  this.next = null;
}

function MyLinkedList() {
  this.head = null;
  this.size = 0;
}

const list = new MyLinkedList();

const node1 = new Node(10);
const node2 = new Node(20);
const node3 = new Node(30);

node1.next = node2;
node2.next = node3;

list.head = node1;
list.size = 3;

let current = list.head;
while (current) {
  console.log(current.val);
  current = current.next;
}
```

Design Linked List

Problem Statement:

Design and implement a custom singly linked list class with the following operations:

Operations:

- `get(index)`: Return the value of the node at the specified index (0-indexed). If index is invalid, return -1.
- `addAtHead(val)`: Insert a node at the beginning of the list.
- `addAtTail(val)`: Append a node at the end of the list.
- `addAtIndex(index, val)`: Insert a node before the index-th node in the list.
- `deleteAtIndex(index)`: Delete the node at the given index.

Approach:

- Use a custom `Node` class that holds `val` and `next` attributes.
- Maintain a `head` pointer to the first node and a `size` to track the list length.
- Each operation ensures index bounds are checked.
- Traverse the list up to the required index using a loop.
- All operations are implemented with time complexity $O(n)$ in the worst case.

```
function Node(val) {
  this.val = val;
  this.next = null;
}

var MyLinkedList = function () {
  this.head = null;
  this.size = 0;
};
```

```
MyLinkedList.prototype.get = function (index) {
  if (index < 0 || index >= this.size) return -1;
  let curr = this.head;
  for (let i = 0; i < index; i++) curr = curr.next;
  return curr.val;
};

MyLinkedList.prototype.addAtHead = function (val) {
  const newNode = new Node(val);
  newNode.next = this.head;
  this.head = newNode;
  this.size++;
};

MyLinkedList.prototype.addAtTail = function (val) {
  const newNode = new Node(val);
  if (!this.head) this.head = newNode;
  else {
    let curr = this.head;
    while (curr.next) curr = curr.next;
    curr.next = newNode;
  }
  this.size++;
};

MyLinkedList.prototype.addAtIndex = function (index, val) {
  if (index < 0 || index > this.size) return;
  if (index === 0) return this.addAtHead(val);
  if (index === this.size) return this.addAtTail(val);
  const newNode = new Node(val);
  let curr = this.head;
  for (let i = 0; i < index - 1; i++) curr = curr.next;
  newNode.next = curr.next;
  curr.next = newNode;
  this.size++;
};

MyLinkedList.prototype.deleteAtIndex = function (index) {
  if (index < 0 || index >= this.size) return;
  if (index === 0) this.head = this.head.next;
  else {
    let curr = this.head;
    for (let i = 0; i < index - 1; i++) curr = curr.next;
    curr.next = curr.next.next;
  }
}
```

```
this.size--;  
};
```

Problem Statement:

Design and implement a custom singly linked list class with the following operations:

Operations:

- `get(index)`: Return the value of the node at the specified index (0-indexed). If index is invalid, return -1.
- `addAtHead(val)`: Insert a node at the beginning of the list.
- `addAtTail(val)`: Append a node at the end of the list.
- `addAtIndex(index, val)`: Insert a node before the index-th node in the list.
- `deleteAtIndex(index)`: Delete the node at the given index.

Approach:

- Use a custom `Node` class that holds `val` and `next` attributes.
- Maintain a `head` pointer to the first node and a `size` to track the list length.
- Each operation ensures index bounds are checked.
- Traverse the list up to the required index using a loop.
- All operations are implemented with time complexity $O(n)$ in the worst case.

```
MyLinkedList.prototype.deleteAtIndex = function (index) {  
  if (index < 0 || index >= this.size) return;  
  if (index === 0) this.head = this.head.next;  
  else {  
    let curr = this.head;  
    for (let i = 0; i < index - 1; i++) curr = curr.next;  
    curr.next = curr.next.next;  
  }  
  this.size--;  
};
```


Find the middle node of a singly linked list

Problem Statement:

Find the middle node of a singly linked list using the slow and fast pointer approach.

Approach:

- Use two pointers: `slow` and `fast`.
- Initialize both at the head of the list.
- Move `slow` one step and `fast` two steps at a time.
- When `fast` reaches the end, `slow` will be at the middle.

Examples:

- **Input:** `[1, 2, 3, 4, 5]`
- **Output:** `[3, 4, 5]`
- **Explanation:** The middle node is node `3`.
- **Input:** `[1, 2, 3, 4, 5, 6]`
- **Output:** `[4, 5, 6]`
- **Explanation:** There are two middle nodes: `3` and `4`. We return the second one.

Constraints:

- The number of nodes in the list is in the range `[1, 100]`.
- `1 <= Node.val <= 100`

Time & Space Complexity:

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(1)`

Use Case:

This technique is efficient with $O(n)$ time and $O(1)$ space, commonly used in problems like finding the start of a loop, checking palindromes, etc.

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var middleNode = function (head) {
    let slow = head;
    let fast = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
};
```

Reverse Linked List

Problem Statement:

Given the head of a singly linked list, reverse the list, and return the reversed list.

Examples:

- Input:** head = [1,2,3,4,5]
Output: [5,4,3,2,1]
- Input:** head = [1,2]
Output: [2,1]

- **Input:** head = []
Output: []

Constraints:

- The number of nodes in the list is in the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Approach:

- Use three pointers: prev, curr, and temp.
- In each step:
 - Save the next node.
 - Point current node's next to previous.
 - Move prev and curr forward.
- Return prev as the new head.

Time and Space Complexity:

- **Time Complexity:** $O(n)$, where n is the number of nodes in the list. We visit each node once.
- **Space Complexity:** $O(1)$, since we reverse the list in-place using a constant number of pointers.

```
var reverseList = function(head) {  
  let prev = null;  
  let curr = head;  
  while (curr) {  
    let temp = curr.next;  
    curr.next = prev;  
    prev = curr;  
    curr = temp;  
  }  
  return prev;  
};
```

Detect Linked List Cycle Using Hash Table

Problem Statement:

Given `head`, the head of a linked list, determine if the linked list has a cycle in it. Return `true` if there is a cycle; otherwise, return `false`.

Examples:

- **Input:** `head = [3,2,0,-4]`, `pos = 1`
Output: `true`
- **Input:** `head = [1,2]`, `pos = 0`
Output: `true`
- **Input:** `head = [1]`, `pos = -1`
Output: `false`

Constraints:

- The number of nodes is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- `pos` is -1 or a valid index in the linked list.

Approach:

- Use a `Set` to track visited nodes.
- While traversing, if we encounter a node already in the set, we've found a cycle.
- If we reach `null`, there's no cycle.

Time and Space Complexity:

- **Time Complexity:** $O(n)$, where n is the number of nodes in the list.
- **Space Complexity:** $O(n)$, in the worst case we store all nodes in a set.

```
var hasCycle = function(head) {  
  let seenNodes = new Set();  
  let curr = head;  
  while (curr !== null) {  
    if (seenNodes.has(curr)) {  
      return true;  
    }  
    seenNodes.add(curr);  
    curr = curr.next;  
  }  
  return false;  
};
```

Linked List Cycle Using Floyd's Cycle Detection

Problem Statement:

Given `head`, the head of a linked list, determine if the linked list has a cycle in it. Return `true` if there is a cycle; otherwise, return `false`.

Examples:

- **Input:** head = [3,2,0,-4], pos = 1
Output: true
- **Input:** head = [1,2], pos = 0
Output: true
- **Input:** head = [1], pos = -1
Output: false

Constraints:

- The number of nodes is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- `pos` is -1 or a valid index in the linked list.

Approach:

- Use Floyd's Cycle Detection (also called the Tortoise and Hare algorithm).
- Use two pointers: slow moves one step, fast moves two steps.
- If there is a cycle, they will eventually meet. If fast or fast.next becomes null, there's no cycle.

Time and Space Complexity:

- **Time Complexity:** $O(n)$, where n is the number of nodes in the list.
- **Space Complexity:** $O(1)$, no extra memory used.

```
var hasCycle = function(head) {  
  if (!head) return false;  
  let slow = head;  
  let fast = head.next;  
  while (slow !== fast) {  
    if (!fast || !fast.next) return false;  
    slow = slow.next;  
    fast = fast.next.next;  
  }  
  return true;  
};
```

Palindrome Linked List

Problem Statement:

Given the `head` of a singly linked list, return `true` if it is a palindrome or `false` otherwise.

Examples:

- **Input:** `head = [1,2,2,1]`
Output: `true`

- **Input:** head = [1,2]
Output: false

Constraints:

- The number of nodes in the list is in the range [1, 10⁵].
- 0 ≤ Node.val ≤ 9

Approach 1:

- Traverse the linked list and store values in an array.
- Check whether the array is a palindrome by comparing elements from start and end moving towards the center.

Time and Space Complexity:

- **Time Complexity:** O(n), where n is the number of nodes.
- **Space Complexity:** O(n), for the array storage.

```
var isPalindrome = function(head) {  
    let arr = [];  
    let curr = head;  
    while (curr !== null) {  
        arr.push(curr.val);  
        curr = curr.next;  
    }  
    let left = 0, right = arr.length - 1;  
    while (left < right) {  
        if (arr[left++] !== arr[right--]) return false;  
    }  
    return true;  
};
```

Intersection of Two Linked Lists

Problem Statement:

Given the heads of two singly linked-lists `headA` and `headB`, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return `null`.

Examples:

- **Input:** `intersectVal = 8`, `listA = [4,1,8,4,5]`, `listB = [5,6,1,8,4,5]`, `skipA = 2`, `skipB = 3`
Output: Intersected at '8'
- **Input:** `intersectVal = 2`, `listA = [1,9,1,2,4]`, `listB = [3,2,4]`, `skipA = 3`, `skipB = 1`
Output: Intersected at '2'
- **Input:** `intersectVal = 0`, `listA = [2,6,4]`, `listB = [1,5]`, `skipA = 3`, `skipB = 2`
Output: No intersection

Constraints:

- The number of nodes in each list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- Lists are guaranteed to be acyclic and maintain their structure.

Approach:

- Store all nodes of `headB` in a `Set`.
- Traverse `headA`; return the node when one exists in the set.
- If no match is found, return `null`.

Time and Space Complexity:

- **Time Complexity:** $O(n + m)$, where n and m are lengths of `listA` and `listB`.
- **Space Complexity:** $O(m)$, storing nodes of `listB` in a set.


```

var getIntersectionNode = function(headA, headB) {
    let set = new Set();
    while (headB) {
        set.add(headB);
        headB = headB.next;
    }
    while (headA) {
        if (set.has(headA)) return headA;
        headA = headA.next;
    }
    return null;
};

```

Remove Linked List Elements

Problem Statement:

Given the head of a linked list and an integer `val`, remove all the nodes of the linked list that have `Node.val == val`, and return the new head.

Examples:

- Input:** head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]
- Input:** head = [], val = 1
Output: []
- Input:** head = [7,7,7,7], val = 7
Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 10^4]$
- $1 \leq \text{Node.val} \leq 50$
- $0 \leq \text{val} \leq 50$

Approach:

- Use a dummy (sentinel) node before the head to simplify edge cases.

- Iterate through the list using a pointer.
- Skip any node whose value matches `val`.
- Return the next of sentinel as the new head.

Time and Space Complexity:

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

```
var removeElements = function(head, val) {  
  let sentinel = new ListNode(0, head);  
  let current = sentinel;  
  while (current && current.next) {  
    if (current.next.val === val) {  
      current.next = current.next.next;  
    } else {  
      current = current.next;  
    }  
  }  
  return sentinel.next;  
};
```

Remove Nth Node From End of Singly Linked List (Two Pass Approach)

Problem Statement:

Given the head of a linked list, remove the `n`th node from the end of the list and return its head.

Examples:

- **Input:** head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]
- **Input:** head = [1], n = 1
Output: []

- **Input:** head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is `sz`.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Approach:

- Use a sentinel (dummy) node before the head to simplify edge cases.
- First, calculate the total length of the list.
- Find the previous node of the one to be deleted using the $\text{length} - n$ formula.
- Update the links to skip the target node.

Time and Space Complexity:

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

```
var removeNthFromEnd = function(head, n) {  
    let sentinel = new ListNode(0, head);  
    let length = 0;  
    let first = head;  
    while (first) {  
        length++;  
        first = first.next;  
    }  
    let prev = sentinel;  
    for (let i = 0; i < length - n; i++) {  
        prev = prev.next;  
    }  
    prev.next = prev.next.next;  
    return sentinel.next;  
};
```

Remove Nth Node From End of Singly Linked List (One Pass Approach)

Problem Statement:

Given the head of a linked list, remove the n th node from the end of the list and return its head.

Approach (Optimized Two-Pointer):

- Use a dummy node before the head to handle edge cases easily.
- Move the first pointer n steps ahead.
- Move both first and second pointers until first reaches the end.
- Now second is just before the node to be deleted. Skip it using `second.next = second.next.next`.

Time and Space Complexity:

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

```
var removeNthFromEnd = function(head, n) {
    let sentinel = new ListNode(0, head);
    let first = sentinel;

    for (let i = 0; i < n; i++) {
        first = first.next;
    }

    let second = sentinel;
    while (first.next) {
        first = first.next;
        second = second.next;
    }

    second.next = second.next.next;
    return sentinel.next;
};
```

Remove Duplicates from Sorted List

Problem Statement:

Given the head of a **sorted linked list**, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Example:

Input: head = [1,1,2]

Output: [1,2]

Input: head = [1,1,2,3,3]

Output: [1,2,3]

Constraints:

- The number of nodes in the list is in the range [0, 300].
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be sorted in ascending order.

Approach:

- Iterate through the linked list with a current pointer.
- For each node, compare its value with the next node.
- If they are equal, skip the next node using `curr.next = curr.next.next`.
- Otherwise, move to the next node.

Time and Space Complexity:

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

```

var deleteDuplicates = function(head) {
  let curr = head;
  while (curr && curr.next) {
    if (curr.val === curr.next.val) {
      curr.next = curr.next.next;
    } else {
      curr = curr.next;
    }
  }
  return head;
};

```

Odd Even Linked List

Problem Statement:

Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

The first node is considered odd (1st position), the second is even (2nd position), and so on.

Example:

Input: head = [1,2,3,4,5]

Output: [1,3,5,2,4]

Input: head = [2,1,3,5,6,4,7]

Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range $[0, 10^4]$.
- $-10^6 \leq \text{Node.val} \leq 10^6$

Approach:

- Maintain two pointers: `odd` starting from the first node and `even` from the second.

- Also store a reference to `evenHead` so that we can attach it to the end of the odd list later.
- While traversing, link odd nodes together and even nodes together alternately.
- Finally, link the last odd node to the head of the even list.

Time and Space Complexity:

- **Time Complexity:** $O(n)$ — Each node is visited exactly once.
- **Space Complexity:** $O(1)$ — In-place manipulation with constant extra space.

```
var oddEvenList = function(head) {  
  if (!head || !head.next) return head;  
  let odd = head;  
  let even = head.next;  
  let evenHead = even;  
  
  while (even && even.next) {  
    odd.next = even.next;  
    odd = odd.next;  
    even.next = odd.next;  
    even = even.next;  
  }  
  odd.next = evenHead;  
  return head;  
};
```

Add Two Numbers Represented by Linked Lists

Problem Statement:

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each node contains a single digit. Add the two numbers and return the result as a linked list.

You may assume the two numbers do not contain any leading zeros, except the number 0 itself.

Examples:

Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807

Input: l1 = [0], l2 = [0]

Output: [0]

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

Approach:

- Initialize a dummy node to build the result list and a `carry` variable set to 0.
- Traverse both linked lists simultaneously. For each node, compute the sum of values plus carry.
- Create a new node with `sum % 10` and update carry to `sum / 10`.
- Continue until both lists are exhausted and carry becomes 0.

Time and Space Complexity:

- **Time Complexity:** $O(n)$ where n is the length of the longer list.
- **Space Complexity:** $O(n)$ for the new list created.


```

var addTwoNumbers = function(l1, l2) {
  let dummy = new ListNode(0);
  let curr = dummy;
  let carry = 0;

  while (l1 || l2 || carry) {
    let sum = (l1?.val || 0) + (l2?.val || 0) + carry;
    carry = Math.floor(sum / 10);
    curr.next = new ListNode(sum % 10);
    curr = curr.next;
    l1 = l1?.next;
    l2 = l2?.next;
  }

  return dummy.next;
};

```

Merge Two Sorted Lists

Problem Statement:

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Examples:

Input: list1 = [1,2,4], list2 = [1,3,4]

Output: [1,1,2,3,4,4]

Input: list1 = [], list2 = []

Output: []

Input: list1 = [], list2 = [0]

Output: [0]

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both `list1` and `list2` are sorted in non-decreasing order.

Approach:

- Use a dummy node to simplify handling of the head.
- Iterate through both lists and append the smaller value node to the merged list.
- Once one list is exhausted, append the remaining part of the other list.
- Return the merged list starting from `dummy.next`.

```
var mergeTwoLists = function(list1, list2) {  
  if (!list1) return list2;  
  if (!list2) return list1;  
  let curr = null;  
  if (list1.val < list2.val) {  
    curr = list1;  
    list1 = list1.next;  
  } else {  
    curr = list2;  
    list2 = list2.next;  
  }  
  let start = curr;  
  while (list1 && list2) {  
    if (list1.val < list2.val) {  
      curr.next = list1;  
      list1 = list1.next;  
    } else {  
      curr.next = list2;  
      list2 = list2.next;  
    }  
    curr = curr.next;  
  }  
  curr.next = list1 || list2;  
  return start;  
};
```

Rotate List

Problem Statement:

Given the head of a linked list, rotate the list to the right by k places.

Examples:

Input: head = [1,2,3,4,5], k = 2

Output: [4,5,1,2,3]

Input: head = [0,1,2], k = 4

Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range [0, 500].
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 * 10^9$

Approach:

- First, compute the length of the list.
- Adjust k using modulo: $k = k \% \text{length}$.
- Use two pointers: move one k steps ahead, then move both until the first reaches the end.
- Rewire the pointers to rotate the list, breaking it at the new tail.

```
var rotateRight = function(head, k) {  
    if (!head || !head.next || k === 0) return head;  
  
    let length = 1;  
    let tail = head;  
    while (tail.next) {  
        tail = tail.next;  
        length++;  
    }  
}
```

```

k = k % length;
if (k === 0) return head;

let stepsToNewHead = length - k;
let prev = null;
let curr = head;
while (stepsToNewHead-- > 0) {
    prev = curr;
    curr = curr.next;
}

prev.next = null;
tail.next = head;
return curr;
};

```

Swap Nodes in Pairs (Iterative approach)

Problem Statement:

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Examples:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Input: head = []

Output: []

Input: head = [1]

Output: [1]

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- $0 \leq \text{Node.val} \leq 100$

Approach:

- Use a dummy node before the head to handle edge cases easily.
- Use pointers to swap pairs by rewiring node connections.
- Iterate through the list two nodes at a time and swap them.

Time and Space Complexity:

- **Time Complexity:** $O(n)$, where n is the number of nodes in the linked list. We visit each node once.
- **Space Complexity:** $O(1)$, constant extra space is used (just a few pointers).

```
var swapPairs = function(head) {  
    if (!head || !head.next) return head;  
  
    let dummy = new ListNode(0);  
    dummy.next = head;  
    let prev = dummy;  
  
    while (prev.next && prev.next.next) {  
        let first = prev.next;  
        let second = first.next;  
  
        first.next = second.next;  
        second.next = first;  
        prev.next = second;  
  
        prev = first;  
    }  
    return dummy.next;  
};
```

Swap Nodes in Pairs (Recursive approach)

Problem Statement:

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Examples:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Input: head = []

Output: []

Input: head = [1]

Output: [1]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- $0 \leq \text{Node.val} \leq 100$

Approach (Recursive):

- Base case: if the list is empty or has only one node, return the head.
- Let `l` be the first node and `r` be the second.
- Call `swapPairs` recursively for the rest of the list starting from `r.next`.
- Set `l.next` to the result of recursive call.
- Make `r` point to `l` and return `r` as the new head.

Time and Space Complexity:

- **Time Complexity:** $O(n)$, where n is the number of nodes in the list.

- **Space Complexity:** $O(n)$ due to recursive call stack.

```
var swapPairs = function(head) {  
  if (!head || !head.next) return head;  
  
  let l = head;  
  let r = head.next;  
  
  l.next = swapPairs(r.next);  
  r.next = l;  
  return r;  
};
```